

AZ500 Lab 10-11 Enabling Just-
In Time Access in VMs,
Microsoft Sentinel

RENE OLUOCH CS-CNS09-25030

Table of Contents

Lab 10: Enabling Just-In-Time Access on VMs	2
Exercise 1: Enabling JIT on your VMs from Azure Virtual Machines	2
Exercise 2: Requesting Access to a JIT-Enabled VM from the Azure Virtual Machine's Connect Page	4
Lab 11: Microsoft Sentinel	6
Exercise 1: Onboarding Microsoft Sentinel.....	6
Exercise 2: Connecting Azure Activity to Sentinel.....	7
Exercise 3: Creating a rule that uses the Azure Activity data connector	11
Task 4: Creating a playbook	11
Task 5: Creating a custom alert and configure a playbook as an automated response	13
Task 6: Invoking an incident and reviewing the associated actions	15
Conclusion	20

Lab 10: Enabling Just-In-Time Access on VMs

Exercise 1: Enabling JIT on your VMs from Azure Virtual Machines

To begin configuring Just-in-Time (JIT) access for my virtual machine, I navigated to the **Virtual machines** blade by entering “virtual machines” into the global search bar and selecting the appropriate result. From the list of available VMs, I selected **myVM**, which had been previously deployed in Lab 08 and was now ready for security enhancements.

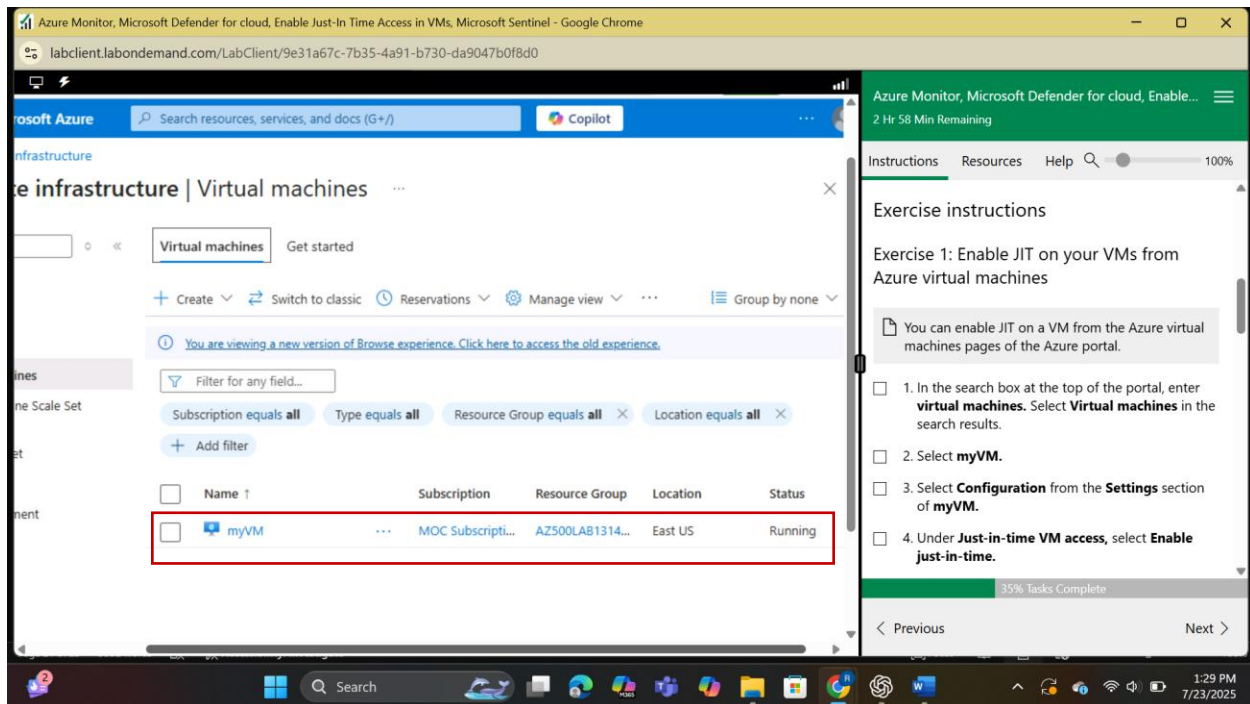


Fig 3.0 Previously deployed, myVM

Once inside the myVM resource page, I scrolled down to the **Settings** section and clicked on **Configuration**. Under the **Just-in-time VM access** option, I located and selected the **Enable just-in-time** link. This action redirected me to the **Microsoft Defender for Cloud** interface, where advanced JIT configurations could be applied. The integration between Azure VM settings and Defender for Cloud made it seamless to enable this feature directly from the VM context.

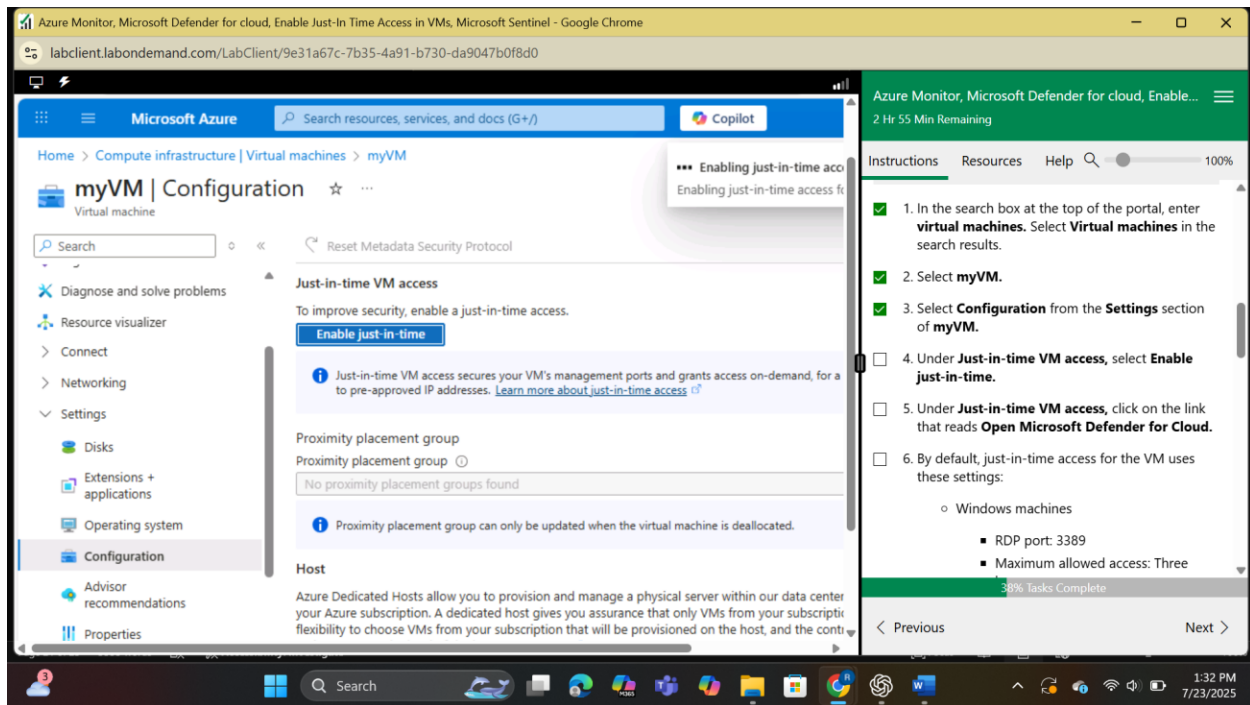


Fig 3.1 Enabling JIT

By default, JIT was configured to protect the most commonly targeted management ports. For this Windows-based machine, that included **TCP port 3389** for Remote Desktop Protocol (RDP). The default rules allowed JIT requests for a maximum of **three hours** and from **any source IP address**, which is useful for initial setup but would typically be tightened in a production scenario.

To view these settings, I went to the **Configured** tab in the JIT configuration panel. From there, I right-clicked on the entry for **myVM** and selected **Edit**. This brought up the configuration interface where I could modify existing port rules or add new ones. Although the defaults were sufficient for this lab, this step demonstrated how additional ports—such as custom application or management ports—could be added and controlled with the same JIT principles. Once reviewed, I selected **Save** to apply the JIT policy to the VM.

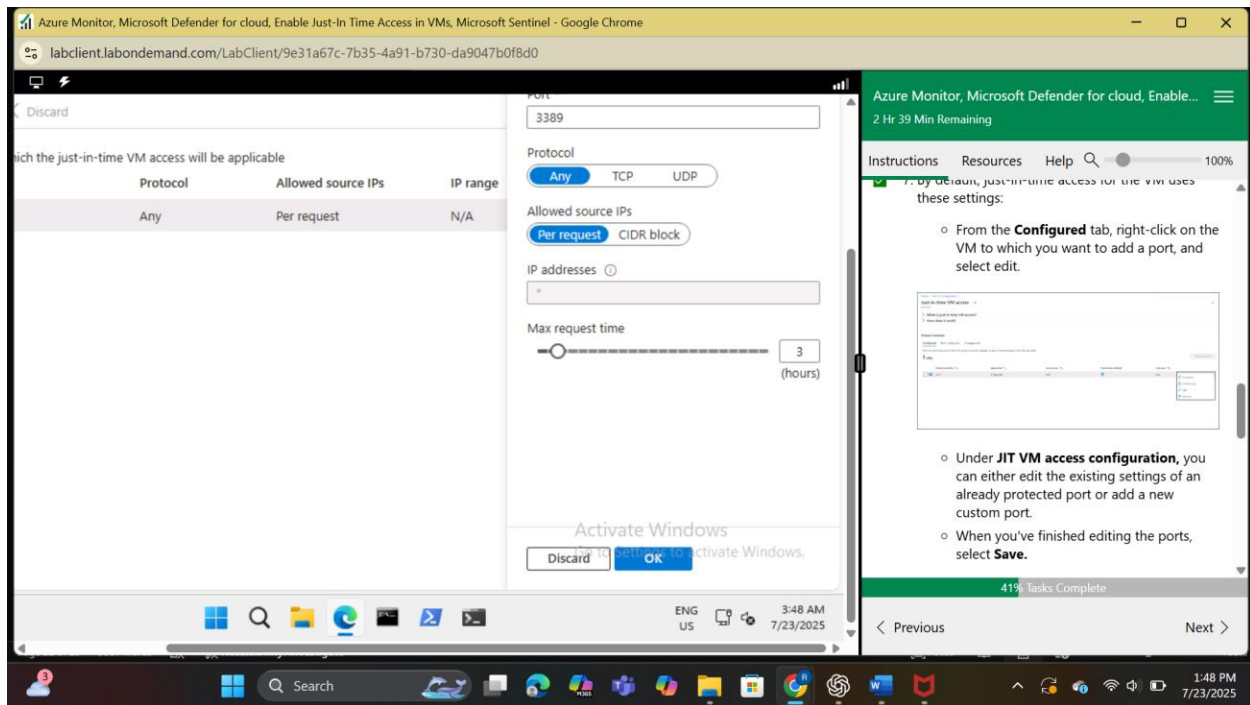


Fig 3.2 Viewing the VM's configurations

At this point, JIT VM access was successfully enabled for myVM. The configuration ensured that inbound traffic to critical ports like RDP would only be allowed when explicitly requested, reducing the attack surface and mitigating risks such as brute-force attacks. This laid the groundwork for secure, time-bound administrative access to the VM, which I would test in the next exercise.

Exercise 2: Requesting Access to a JIT-Enabled VM from the Azure Virtual Machine's Connect Page

After successfully enabling Just-in-Time (JIT) VM access on **myVM**, I proceeded to validate the configuration by requesting access through the Azure portal. This step ensures that even though JIT restricts open port access by default, administrators can still securely connect to VMs when necessary by issuing approved access requests.

To begin, I returned to the **Virtual machines** blade in the Azure portal and selected **myVM** from the list. Once inside the VM overview page, I clicked on **Connect** in the top menu — the standard way to initiate a Remote Desktop session for Windows VMs. As expected, Azure immediately detected that JIT was enabled for this VM.

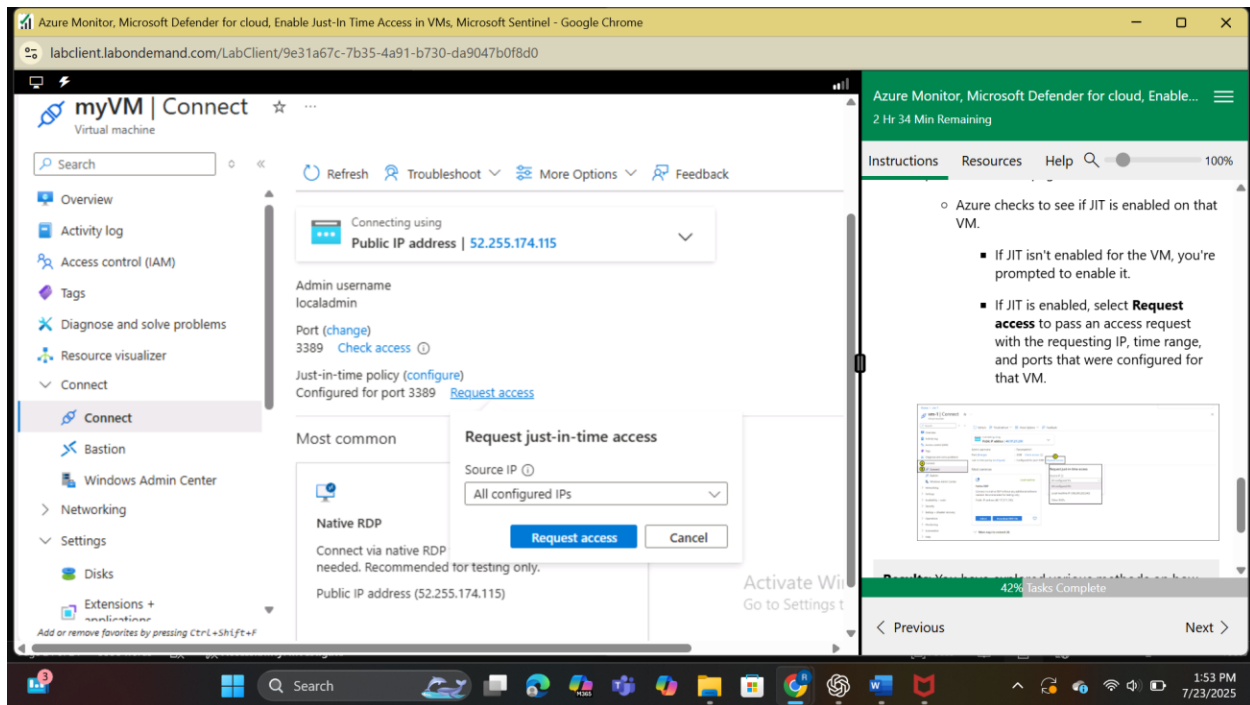


Fig 3.3 Jit enabled on the vm

Instead of allowing a direct connection, the portal prompted me to **Request access**, which launched a configuration panel. Here, I confirmed the parameters of my access request. The portal pre-filled the **RDP (3389)** port, and I left the access duration at the default of **3 hours**. I also left the **source IP address setting** as "Any" to keep the configuration consistent with the lab's defaults.

Once I reviewed these settings, I clicked **Request access**. Azure Defender for Cloud processed the request, and within seconds, the firewall rules were temporarily opened to allow the connection from my current IP address. This confirmed that JIT access was not only enabled but functional, providing secure, on-demand administrative access while minimizing exposure.

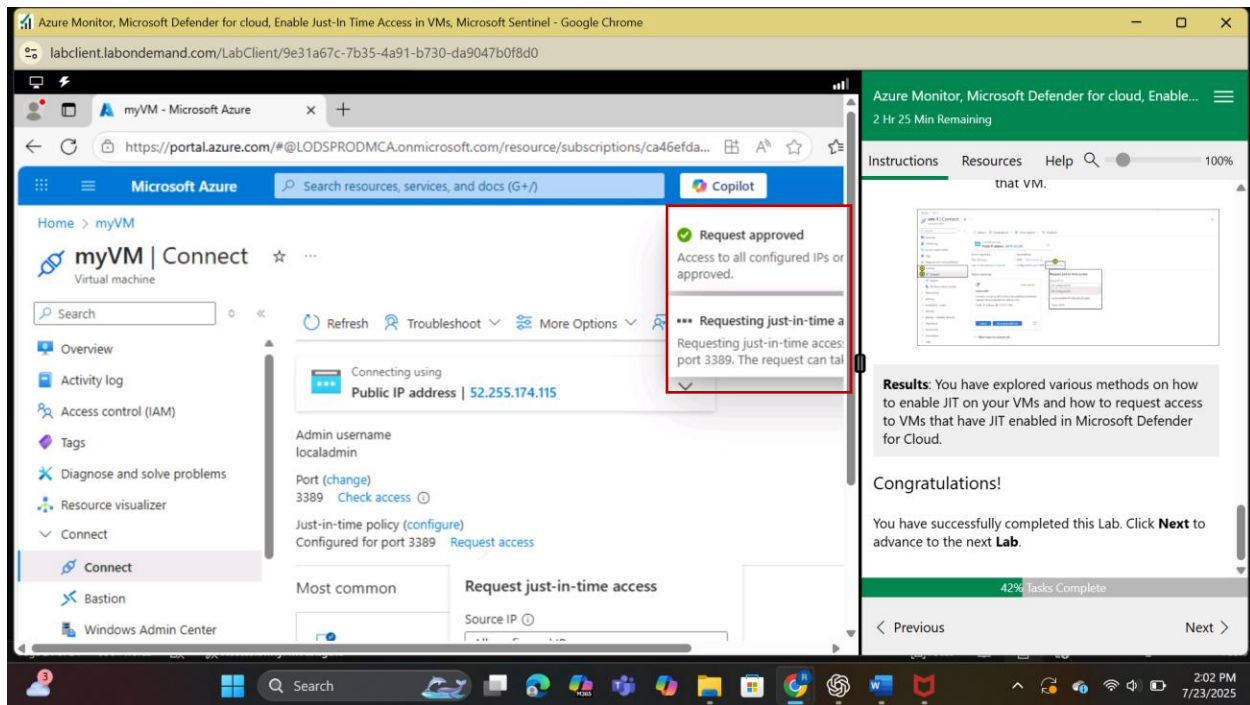


Fig 3.4 Request and approval of access to all configured ip

With this test, I verified that the JIT configuration worked as intended: by default, critical ports remain closed, and temporary access can be granted in a controlled and time-bound manner. This effectively reduces the window of opportunity for potential attacks while still supporting operational needs.

By the end of this exercise, I had successfully demonstrated the process of both enabling and utilizing JIT VM access — a powerful security measure that aligns with best practices for privileged access management in Azure.

Lab 11: Microsoft Sentinel

In this lab, I implemented Microsoft Sentinel to create a proof of concept for threat detection and automated response. The goal was to start collecting data from Azure Activity and Microsoft Defender for Cloud, add both built-in and custom alerts, and review how playbooks can automate responses to incidents. All resources were created in the East US region.

Exercise 1: Onboarding Microsoft Sentinel

To begin, in the Azure portal, I searched for "Microsoft Sentinel" in the top search bar and selected it from the services list. On the Microsoft Sentinel blade, I clicked "+ Create" to add Sentinel to a workspace. I selected the Log Analytics workspace that I had created during the Azure Monitor lab and then clicked Add. I ensured that the workspace met Sentinel's

requirements, such as not being created by Microsoft Defender for Cloud, which is a limitation for compatibility.

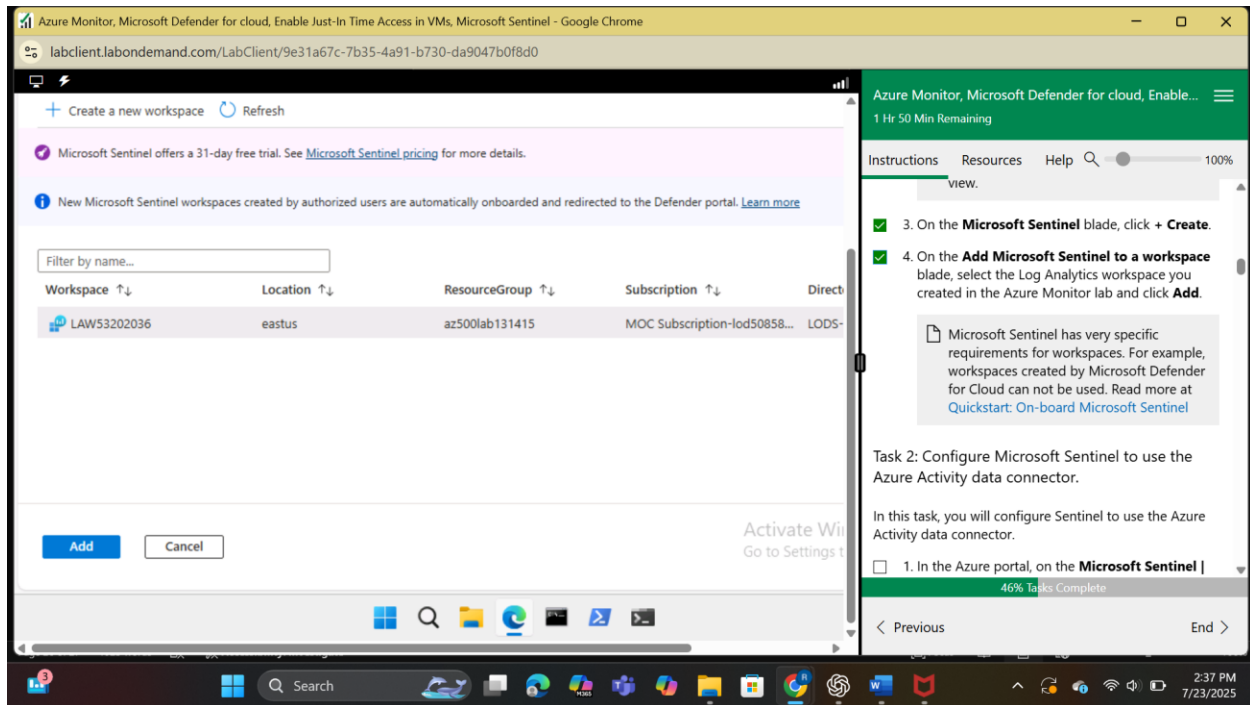


Fig 3.5 Adding Log Analytics workspace created earlier

Exercise 2: Connecting Azure Activity to Sentinel

After onboarding, I configured Microsoft Sentinel to use the Azure Activity data connector. From the Sentinel Overview blade, I clicked Content hub under the Content management section. In the Content hub, I searched for "Azure Activity," reviewed the description, and clicked Install.

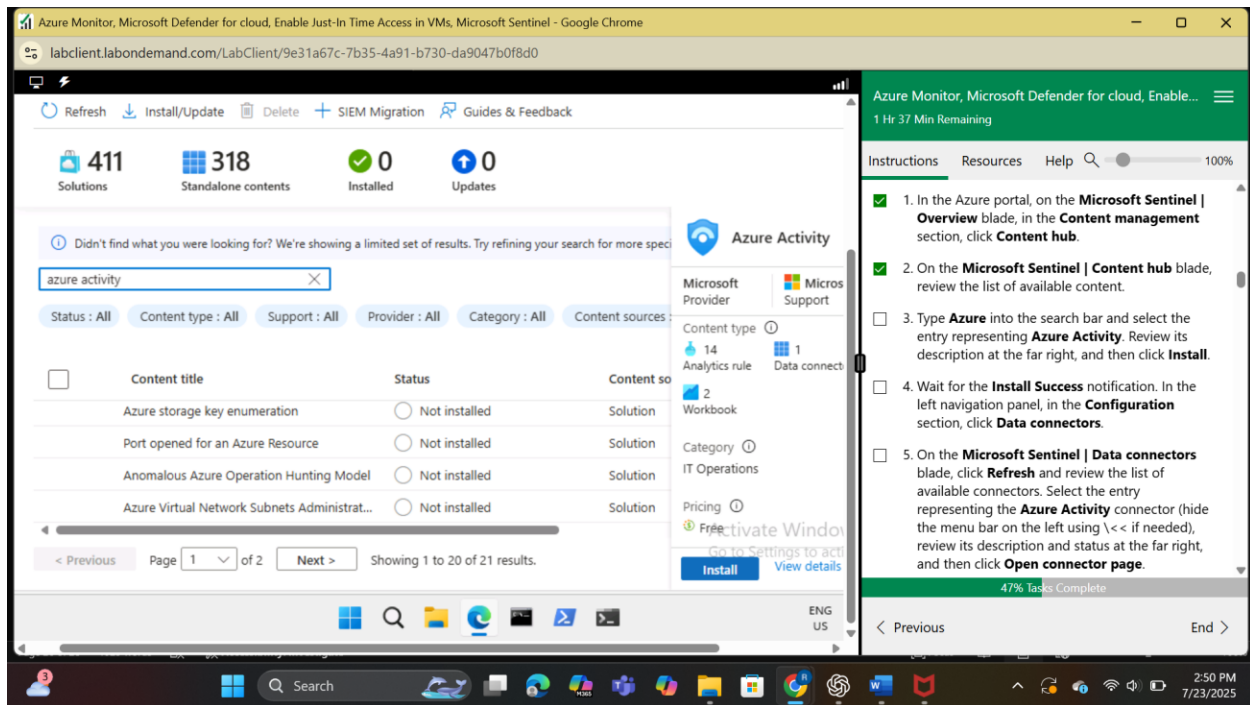


Fig 3.6 Installing azure activity

Once installed, I navigated to the Data connectors section and refreshed the page to see the updated list. I selected the Azure Activity connector and clicked Open connector page. On the Azure Activity connector page, under the Instructions tab, I skipped step 1 since the Disconnect All button was greyed out, indicating no legacy connectors were used. In step 2, I clicked "Launch the Azure Policy Assignment wizard" and on the Basics tab, I set the scope by selecting my subscription without choosing a resource group. I clicked Next twice to reach the Parameters tab and then selected my Log Analytics workspace.

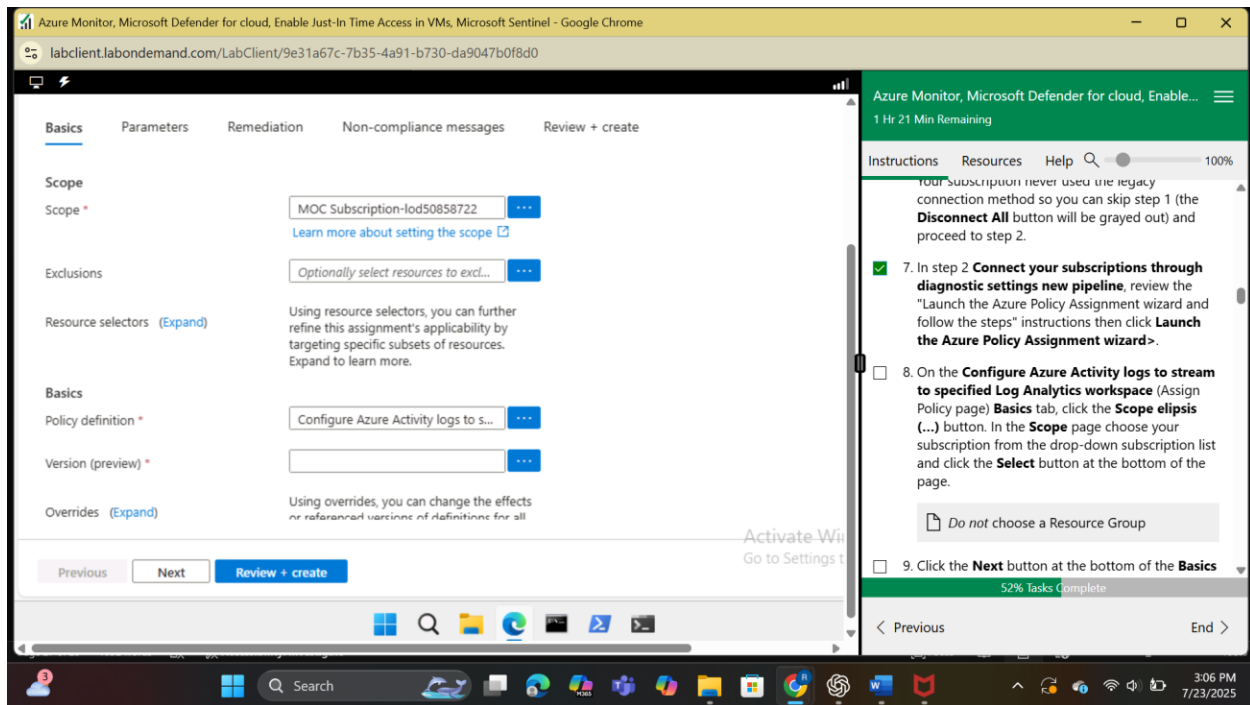


Fig 3.7 Filling in the required information on the Azure Policy Assignment wizard

On the Remediation tab, I checked the box to create a remediation task and set the system-assigned identity location to East US. I skipped the optional non-compliance message and clicked Review + Create, then Create. I verified successful creation of the policy assignment, role assignments, and remediation task by checking the Azure notifications bell icon. I waited for data to be received, which was confirmed by the graph appearing on the Azure Activity connector pane after refreshing.

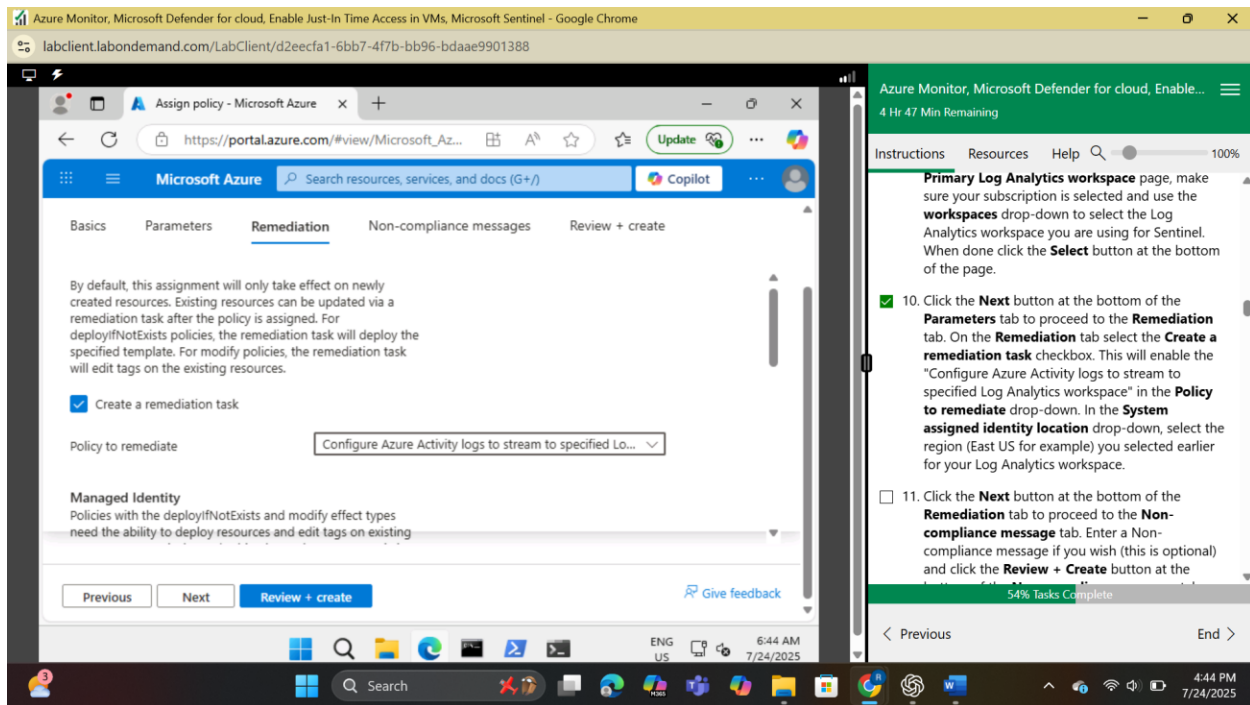


Fig 3.8 Filling in required information on remediation pane

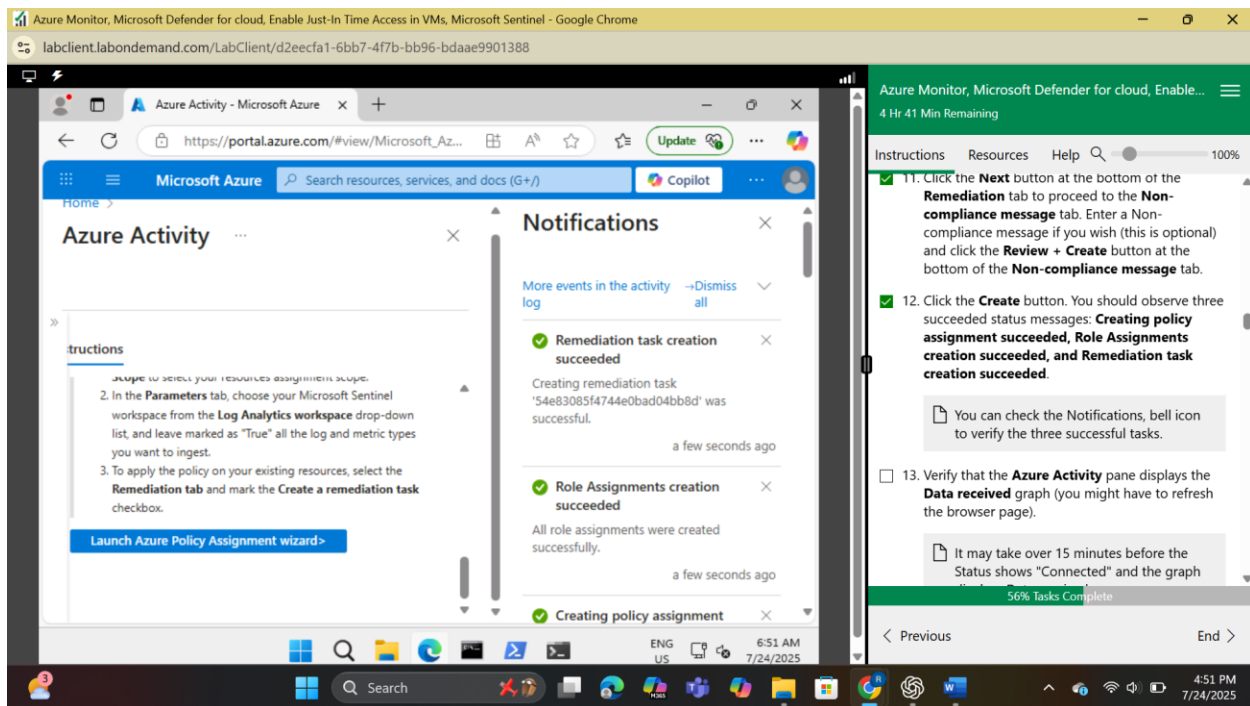


Fig 3.9 Successfully creating the assignments

Exercise 3: Creating a rule that uses the Azure Activity data connector

Next, I created a rule using the Azure Activity data connector. From the Microsoft Sentinel configuration blade, I clicked on Analytics, selected the Rule templates tab, and searched for "Suspicious." I chose the rule titled "Suspicious number of resource creation or deployment," which is associated with Azure Activity, and clicked Create rule. I accepted all the default settings in the General, Set rule logic, Incident settings, and Automated response tabs. On the final Review and create tab, I clicked Save, activating the rule.

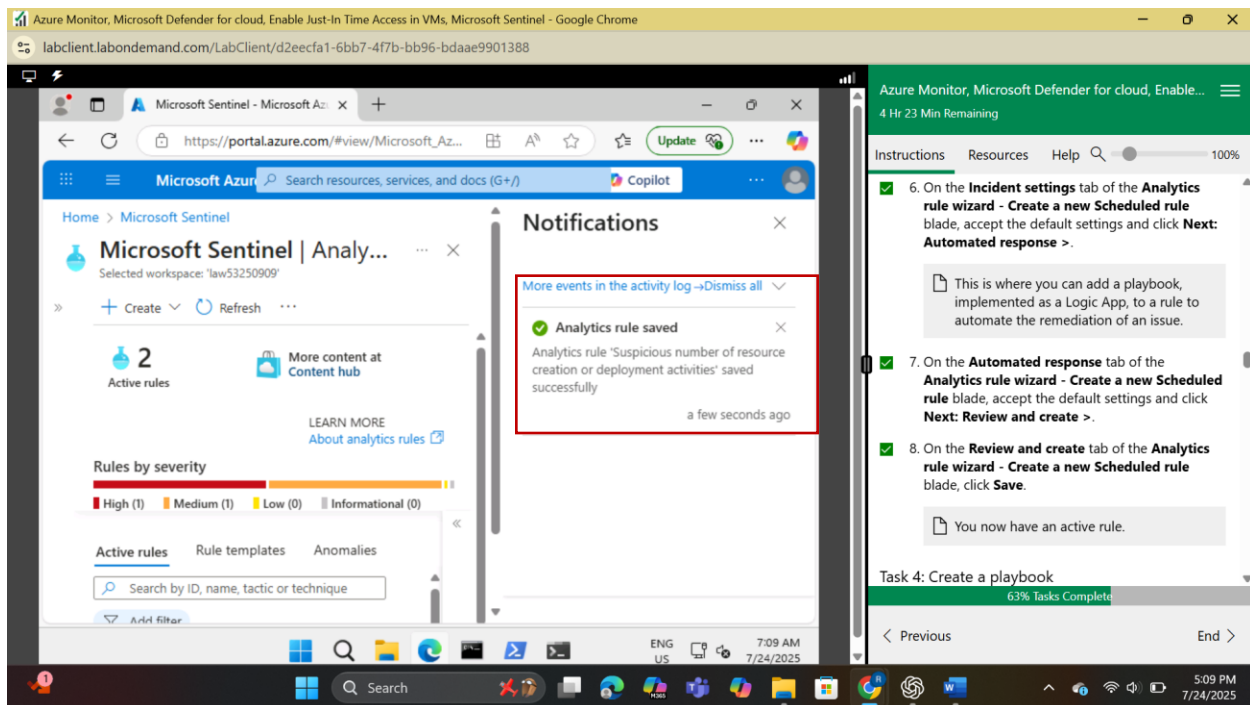


Fig 4.0 Successfully creating a new scheduled rule

Task 4: Creating a playbook

To automate responses, I created a playbook. In the Azure portal, I searched for "Deploy a custom template" and selected "Build your own template in the editor." In the template editor, I clicked Load file and uploaded the "changeincidentseverity.json" file located in the provided lab path. After loading, I saved the template.

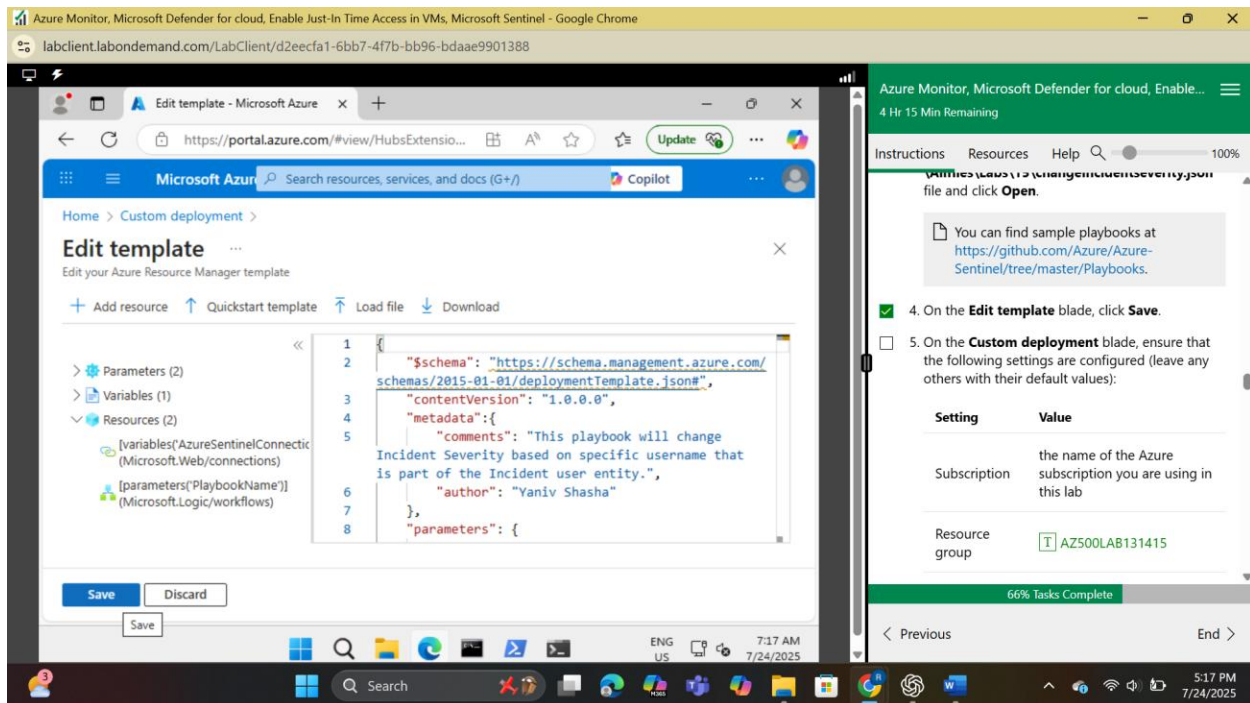


Fig 4.1 Editing json file

On the Custom deployment blade, I set the subscription to the one used for the lab, chose the resource group AZ500LAB131415, and set the location to East US. I named the playbook "Change-Incident-Severity" and entered my email address. I reviewed the details and clicked Create.

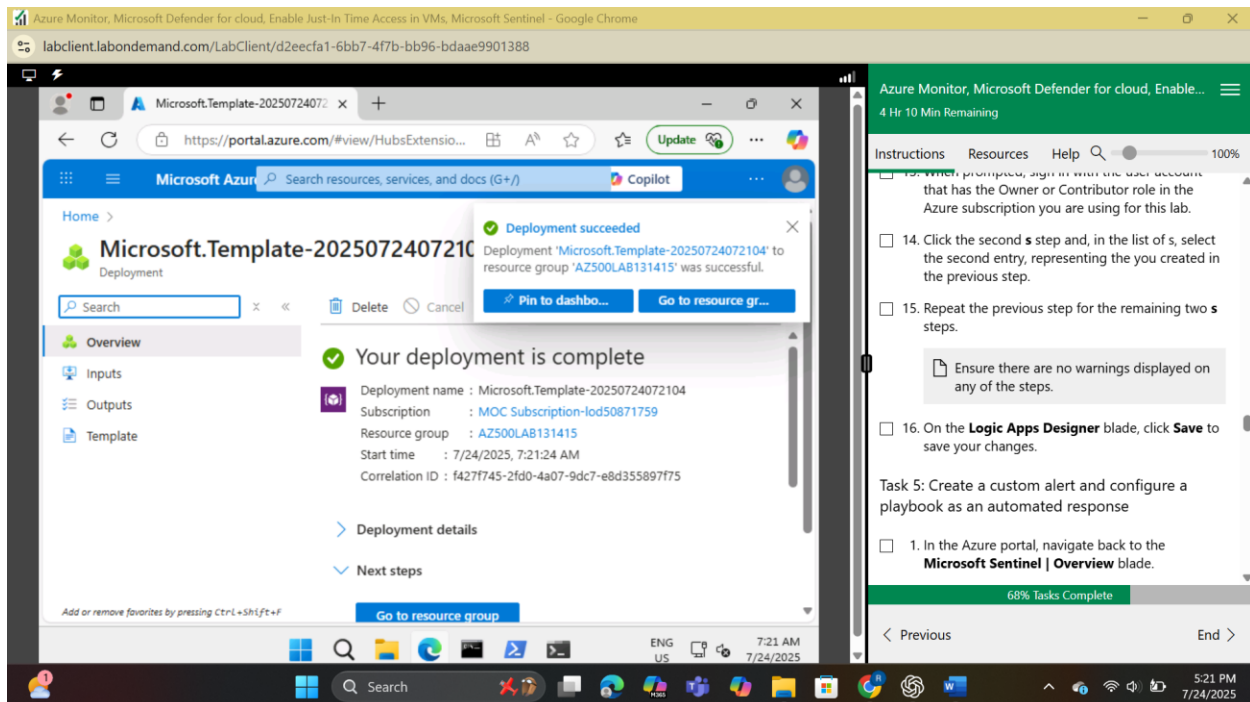


Fig 4.2 Successful deployment

After deployment, I navigated to the AZ500LAB131415 resource group and selected the Change-Incident-Severity logic app. On its blade, I clicked Edit. The Logic App Designer showed warning symbols for each step, indicating missing configurations. I clicked on the first step, added a new connection, verified my Azure AD tenant, and signed in. I repeated this process for the remaining three steps to resolve all warnings. Once all steps were configured correctly, I saved the logic app.

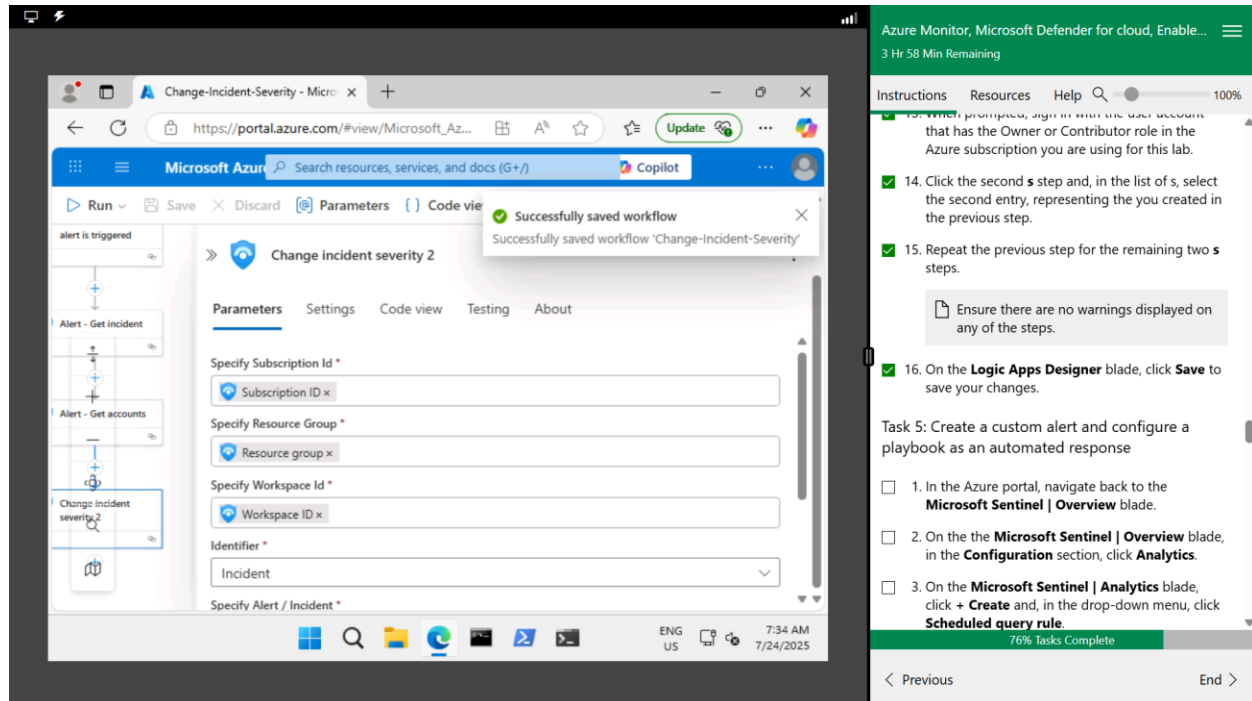


Fig 4.3 Successfully saving the workflow

Task 5: Creating a custom alert and configure a playbook as an automated response

Following this, I created a custom alert and linked the playbook as an automated response. I returned to the Microsoft Sentinel Overview blade, selected Analytics under Configuration, and clicked + Create, choosing Scheduled query rule. On the General tab, I named the rule "Playbook Demo" and selected "Initial Access" as the tactic. On the Set rule logic tab, I entered the following KQL query:

```
AzureActivity | where ResourceProviderValue =~ "Microsoft.Security" | where  
OperationNameValue =~ "Microsoft.Security/locations/jitNetworkAccessPolicies/delete"
```

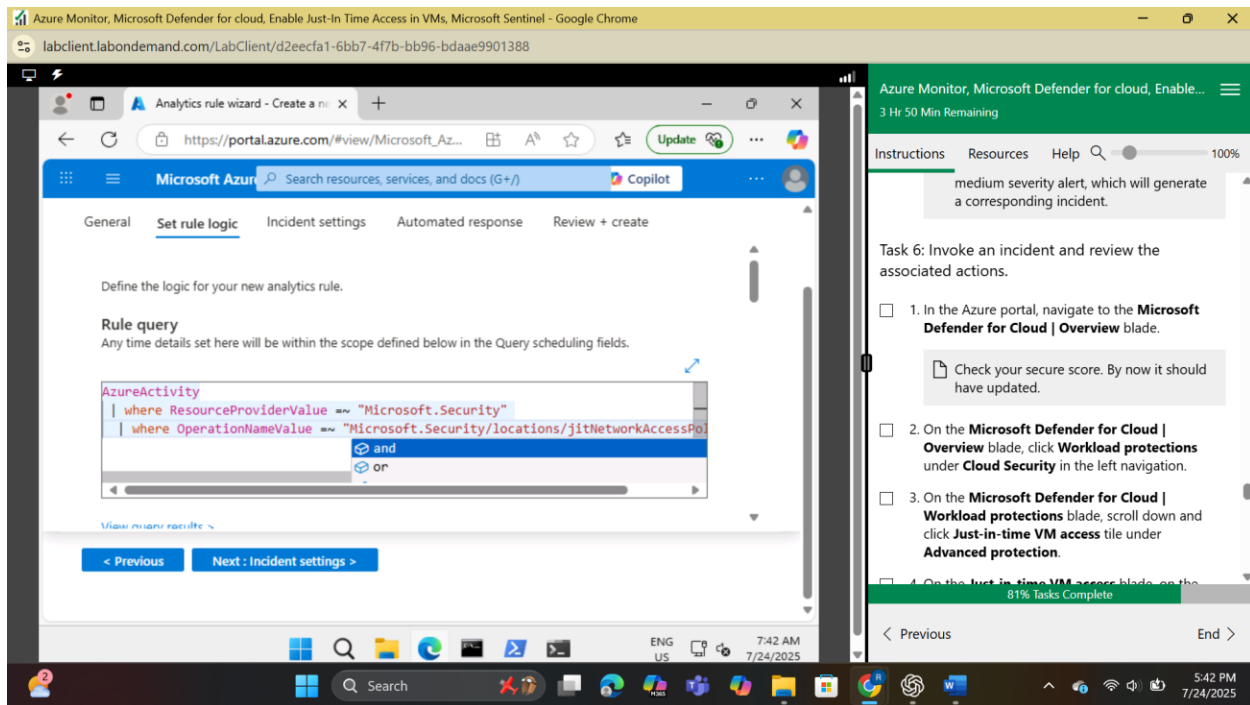



Fig 4.4 Entering KQL query on the set rule tab

This query detects deletion of Just-in-time VM access policies. I set the query to run every five minutes and kept all other default settings, clicking through to the Automated response tab. There, I added a new automation rule, named it "Run Change-Severity Playbook," and set the trigger to "When alert is created." I clicked "Manage playbook permissions," selected the AZ500LAB131415 resource group, and clicked Apply. I then selected the Change-Incident-Severity logic app and completed the setup by clicking Apply and then Save.

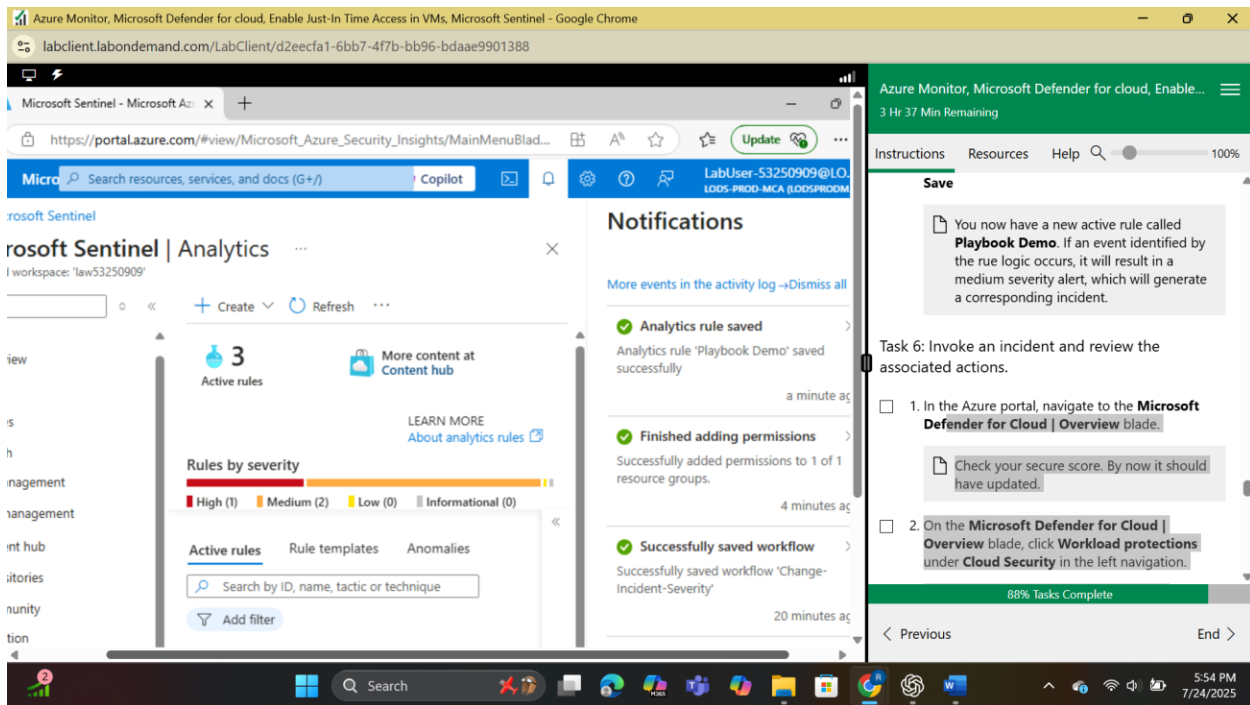


Fig 4.5 Successfully creating the analytics rule

Task 6: Invoking an incident and reviewing the associated actions

To test the incident, I went to Microsoft Defender for Cloud and reviewed the secure score. Under Workload protections, I selected Just-in-time VM access. On the row corresponding to the "myVM" virtual machine, I clicked the ellipsis button, chose Remove, and confirmed the action.

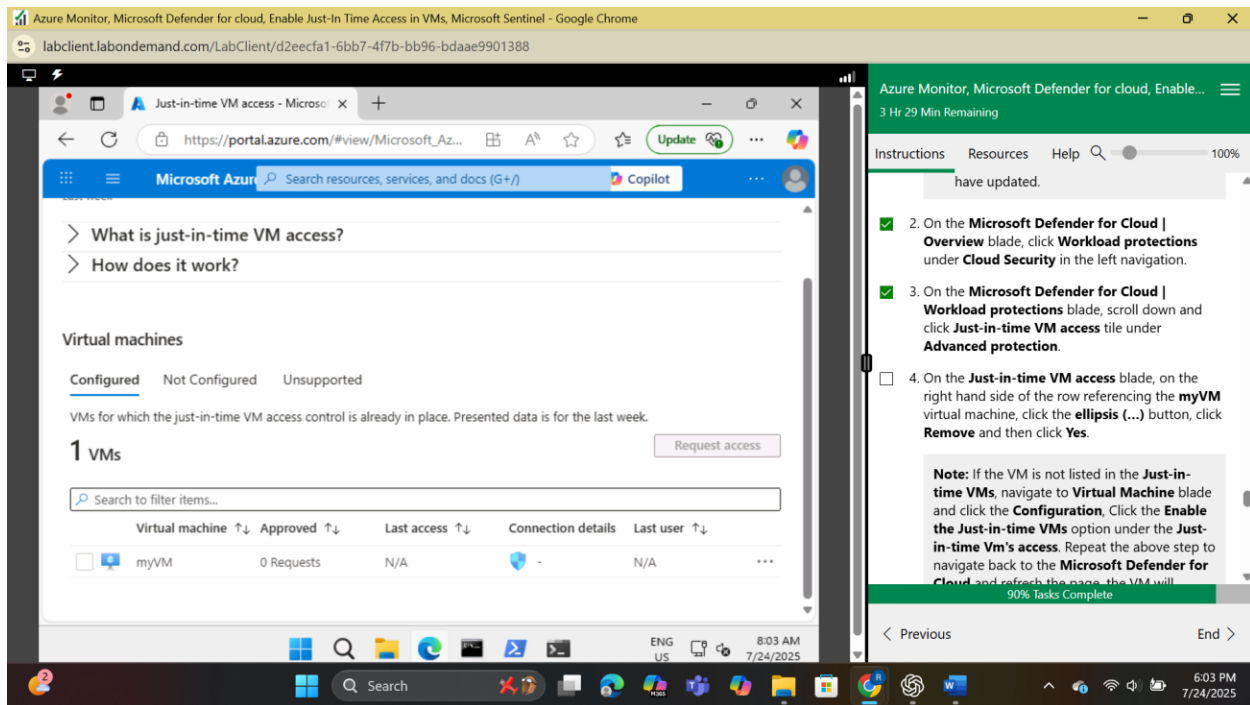


Fig 4.6 Removing myVM

I verified the action by checking the Activity log, where a “Delete JIT Network Access Policies” event appeared.

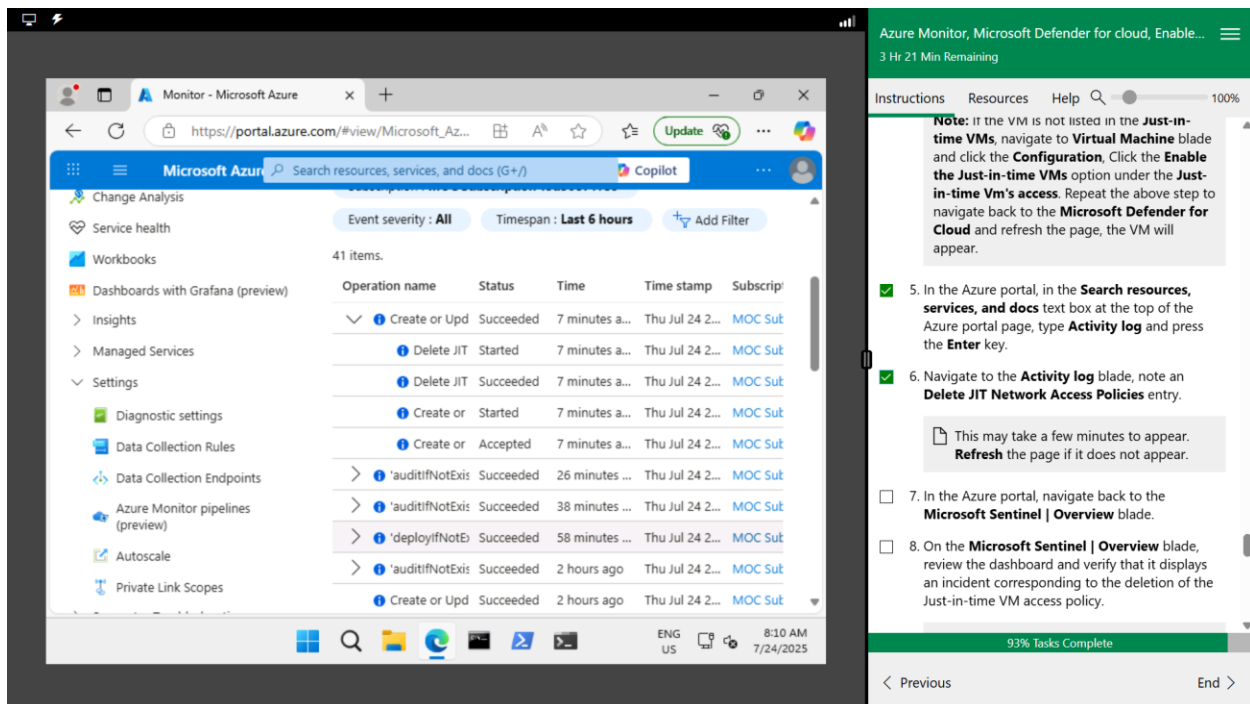


Fig 4.7 Checking the activity log

I then returned to the Microsoft Sentinel Overview blade. Within a few minutes, an incident appeared on the dashboard and under the Incidents section with a or high severity level. I also reviewed the Playbooks blade, which showed successful and failed executions, and confirmed that the triggered alert had invoked the playbook successfully.

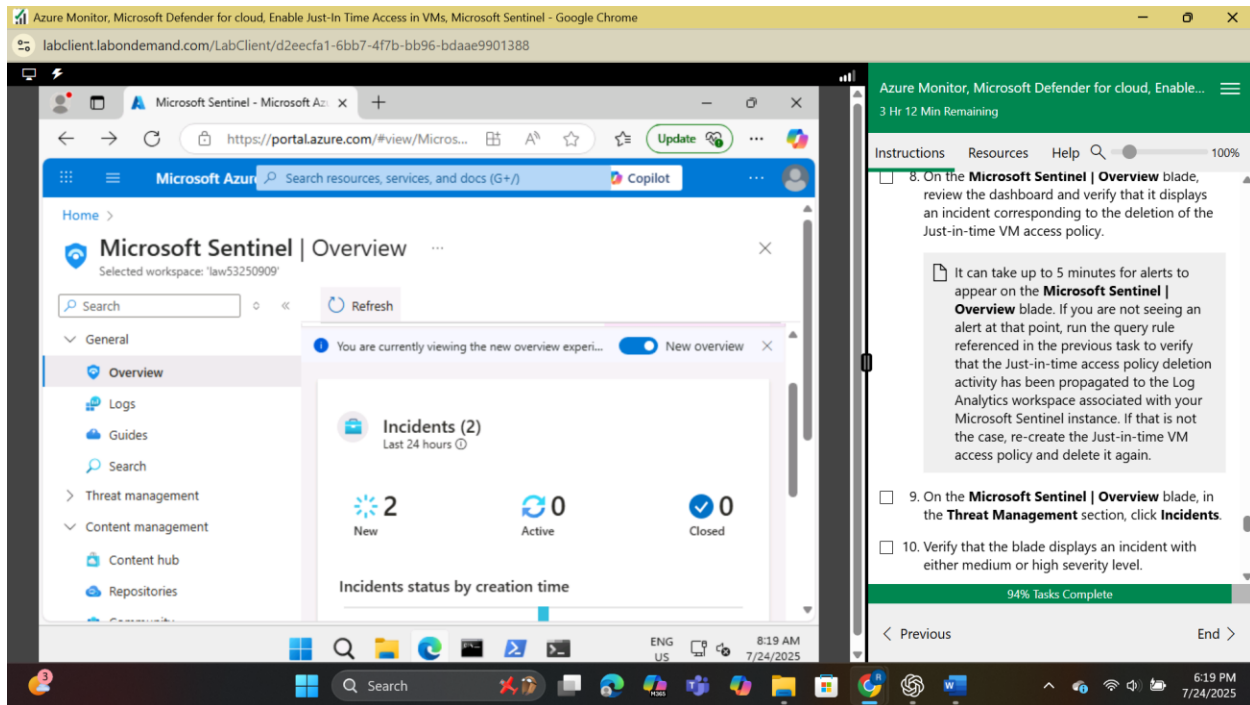


Fig 4.8 Microsoft Sentinel showing incidents with high level severity

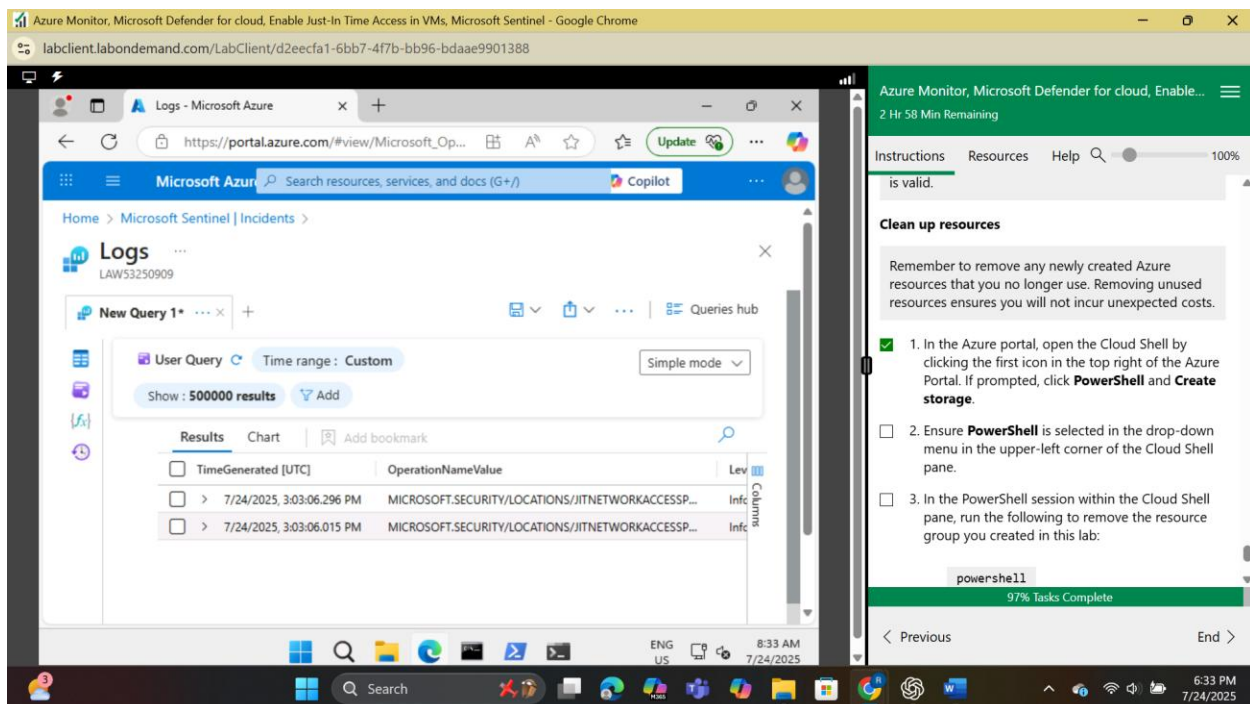


Fig 4.9 Analytic info on the deleted JIT myVM

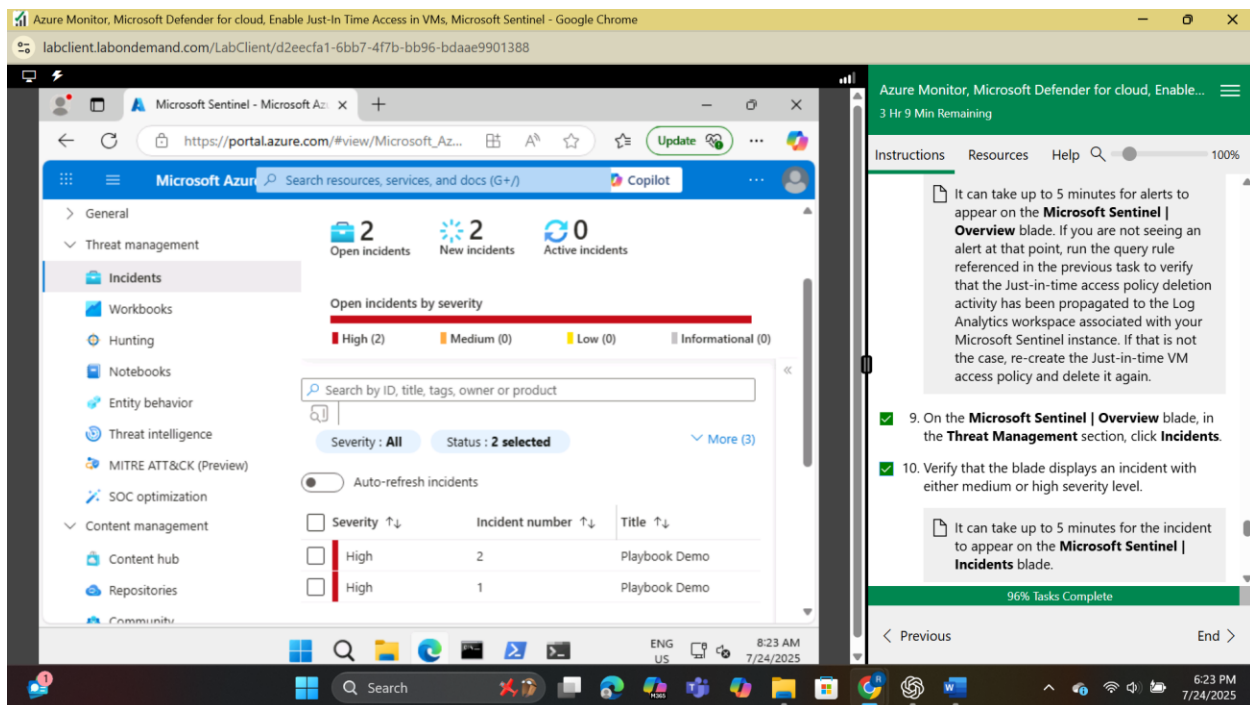


Fig 5.0 The incident section showing activities with high severity

Finally, I cleaned up the resources used in the lab. I opened the Cloud Shell in Azure, selected PowerShell, and entered the following command to delete the resource group:

```
Remove-AzResourceGroup -Name "AZ500LAB131415" -Force -AsJob
```

After confirming the job started, I closed the Cloud Shell pane.

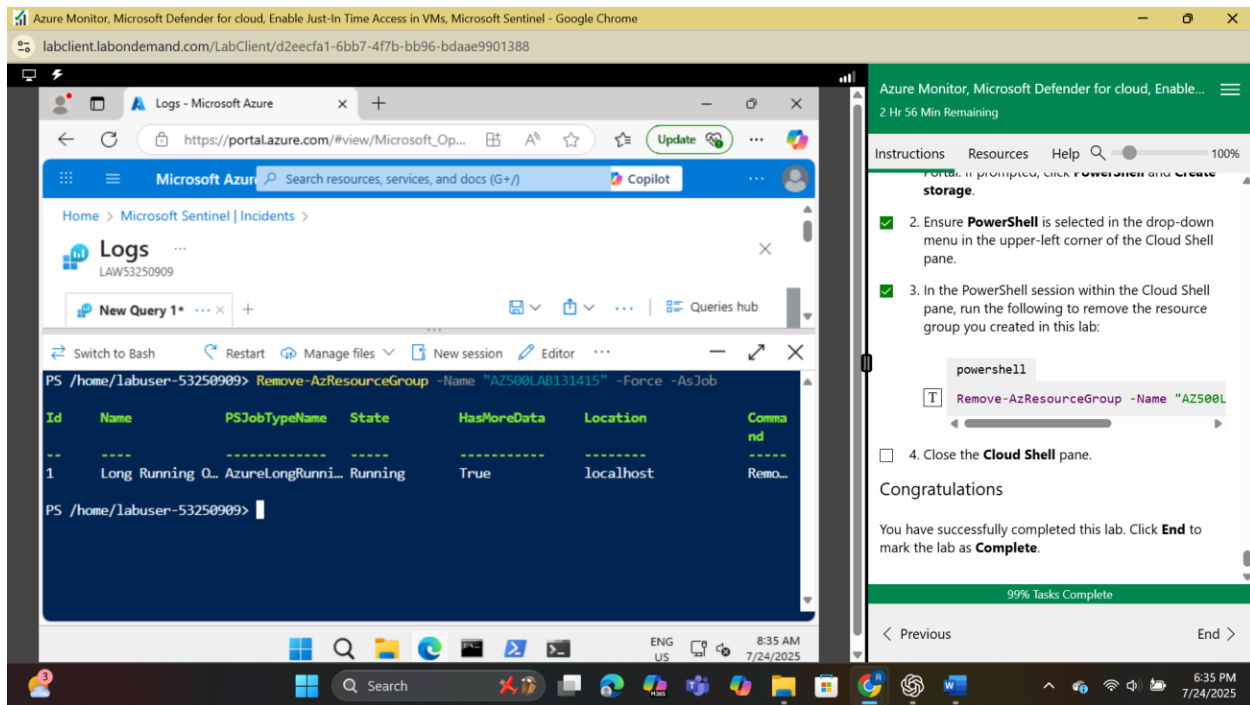


Fig 5.1 Cleaning up the resource

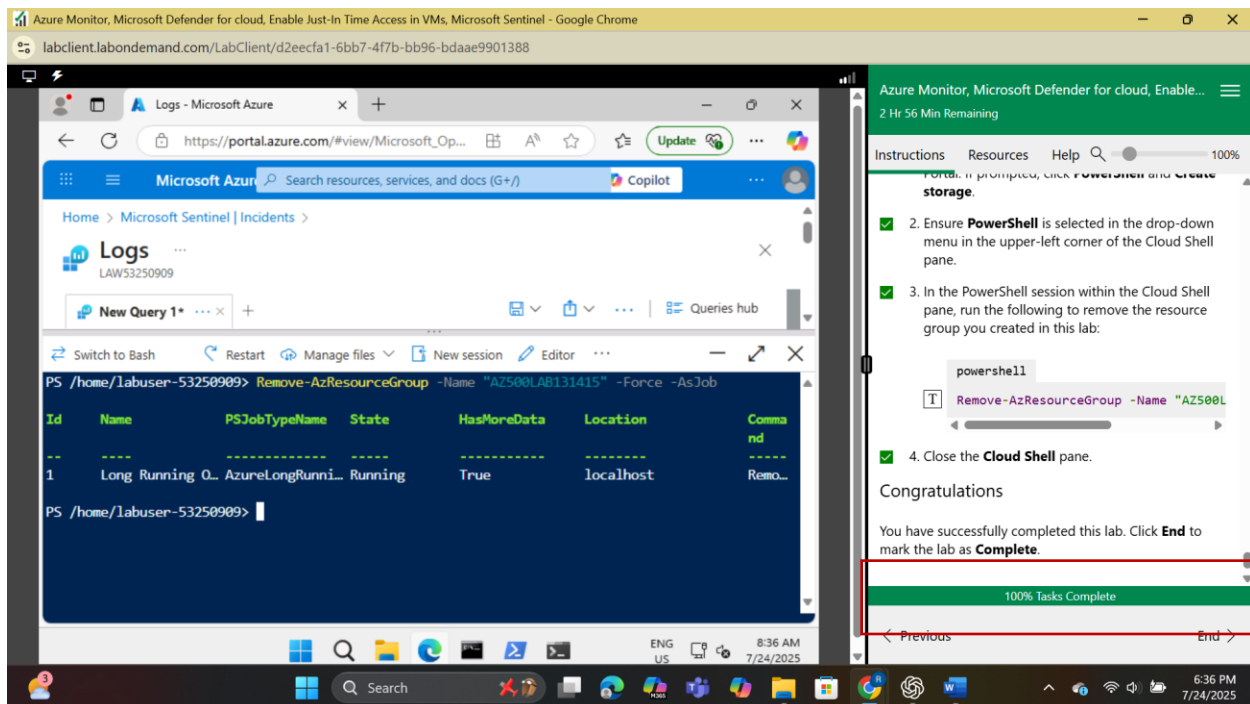


Fig 5.2 Lab completion

This concluded Lab 11. I successfully created a Microsoft Sentinel workspace, connected it to Azure Activity logs, configured both a playbook and custom alert to respond to Just-in-time VM access policy deletions, and validated the end-to-end response flow.

Conclusion

Throughout Labs 8 to 11, I applied a comprehensive set of security tools to protect and monitor Azure resources effectively. I deployed and secured a virtual machine, enabled advanced protection with Microsoft Defender for Servers Plan 2, configured just-in-time access to limit unnecessary exposure, and implemented Microsoft Sentinel to detect, respond to, and automate alerts based on real-time activity. These labs provided a well-rounded, practical approach to modern cloud security operations, reinforcing the importance of visibility, control, and automation in safeguarding cloud infrastructure.